

Explicación pedagógica del pipeline

Pensión 65 — Adopción de Billetera Digital

*Una guía ordenada para entender el archivo
scripts/script_completo/script.py*

Documento explicativo del pipeline de investigación reproducible para el paper sobre el efecto de Pensión 65 en la adopción de billeteras digitales en el Perú, usando datos de la ENAHO 2024.

Contenido

1. Introducción: ¿qué hace este script?
2. ¿Por qué importa la reproducibilidad?
3. Arquitectura general: las cinco fases
4. Fase 0 — Descarga automática desde el INEI
5. Fase 1 — Preprocesamiento de la ENAHO
6. Fase 2 — Limpieza para regresión discontinua
7. Fase 3 — Estimación principal del modelo
8. Fase 4 — Pruebas de robustez
9. Fase 5 — Tablas y figuras del paper
10. Cómo correr el pipeline
11. Glosario de conceptos clave
12. Conclusiones

1. Introducción: ¿qué hace este script?

Empecemos por lo más importante. Este documento explica, paso a paso, el archivo `scripts/script_completo/script.py`, que es el corazón computacional de un proyecto de investigación econométrica sobre los **determinantes de la adopción de billetera digital en el Perú**, con énfasis en el efecto del programa **Pensión 65**.

Si tuviéramos que resumir lo que hace el script en una sola oración:

■ En una oración

Toma datos crudos de la **ENAH0 2024** directamente desde el servidor del INEI, los procesa, estima un modelo de **regresión discontinua (RDD)** para medir el efecto de cumplir 65 años sobre la tenencia y uso de billetera digital, ejecuta nueve pruebas de robustez, y genera todas las tablas y figuras del paper académico final.

1.1 ¿Por qué un solo archivo?

Tradicionalmente, un proyecto de investigación tiene cinco, ocho o más scripts encadenados. Para reproducir los resultados, otro investigador debe correrlos en el orden correcto, configurando rutas, descargando datos manualmente, y rezando para que nada se rompa. Eso es frágil.

Este script hace lo opuesto: **todo en un solo archivo, un solo comando**. La filosofía detrás es la **reproducibilidad total**:

- Cualquier persona puede clonar el repositorio.
- Correr `python script.py`.
- Obtener exactamente los mismos resultados que el autor original.

Sin pasos manuales. Sin configuración. Sin descargas externas previas. Sin '*¿en qué orden corro estos archivos?*'.

2. ¿Por qué importa la reproducibilidad?

En investigación científica, un resultado que no se puede reproducir no es un aporte: es una afirmación. La diferencia entre ciencia y opinión es justamente que la ciencia se puede verificar.

2.1 Los tres pilares de un pipeline reproducible

Este script está construido sobre tres pilares concretos:

Pilar	Implementación en el script
Reproducible	Semilla aleatoria fija (RANDOM_SEED = 42) para que las permutaciones y bootstraps den siempre el mismo resultado.
Automático	Auto-instalación de dependencias y descarga directa de datos desde el INEI, sin intervención manual.
Aislado	Todas las rutas se calculan desde la ubicación del propio script (<code>__file__</code>) usando <code>pathlib.Path</code> . Cero rutas absolutas.

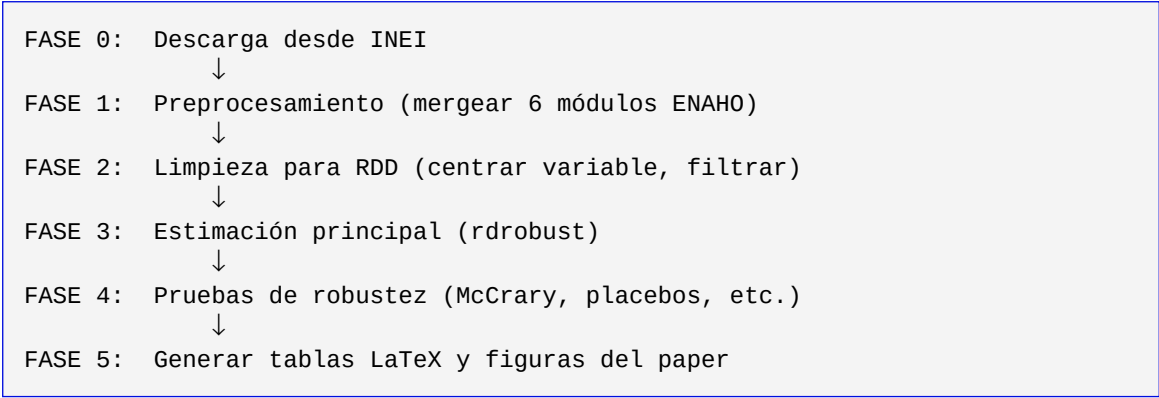
■ ■ Regla de oro

Si en el código encontramos algo como `C:/Users/Pedro/...`, ya estamos perdidos: ese script solo funciona en la computadora de Pedro. La regla de oro es que **todas las rutas deben ser relativas al proyecto**, nunca absolutas a una máquina específica.

3. Arquitectura general: las cinco fases

El script está organizado de manera **secuencial** en cinco fases. Cada fase es una función independiente con un único trabajo. Si algo falla en la fase 3, sabes exactamente dónde mirar. Si necesitas modificar la limpieza, solo tocas la fase 2. Eso se llama **separación de responsabilidades**.

3.1 Diagrama del flujo



3.2 ¿Cómo se conectan las fases?

Cada fase produce **archivos en disco** que la siguiente lee. Esto tiene una ventaja enorme: si una fase ya corrió, no necesitas recorrerla de nuevo. Puedes ejecutar solo las fases que cambiaron, y debuggear más rápido.

Fase	Lee	Escribe
0	(internet)	data/clean/_raw_extracted/*.csv
1	_raw_extracted/*.csv	data/clean/enaho_2024_clean.csv
2	enaho_2024_clean.csv	data/clean/clean_data.csv
3	clean_data.csv	data/clean/main_results.csv
4	clean_data.csv	data/clean/robustness_results.csv
5	Los CSVs anteriores	paper/tables/*.tex, paper/figures/*

4. Fase 0 — Descarga automática desde el INEI

4.1 ¿Qué problema resuelve?

Antes de esta fase, la única forma de obtener los datos era: ir a la web del INEI, hacer clic en cada uno de los seis módulos, descargar los ZIP a mano, y subirlos al proyecto. Eso es **manual, frágil y no reproducible**.

La fase 0 elimina ese paso manual. Descarga directamente desde los servidores del INEI usando el patrón de URL público:

```
https://proyectos.inei.gob.pe/iinei/srienaho/descarga/CSV/  
966-Modulo{XX}.zip
```

Donde **966** es el código de la encuesta ENAHO Anual 2024, y **XX** es el código de cada módulo.

4.2 Los seis módulos descargados

Código	Módulo (nombre INEI)	Archivo CSV
01	Características de la Vivienda	Enaho01-2024-100.csv
02	Miembros del Hogar	Enaho01-2024-200.csv
03	Educación	Enaho01A-2024-300.csv
05	Empleo + Billetera Digital	Enaho01a-2024-500.csv
18	Equipamiento del Hogar	Enaho01-2024-612.csv
34	Sumaria (variables calculadas)	Sumaria-2024.csv

4.3 Cómo funciona técnicamente

El script usa `urllib.request` (que viene incluido en Python, sin librerías externas) para descargar cada ZIP, y `zipfile` para extraer únicamente el CSV que necesita el pipeline:

```
url = f"{INEI_BASE_URL}/{INEI_SURVEY_CODE}-Modulo{code}.zip"  
req = urllib.request.Request(url, headers={"User-Agent": "Mozilla/5.0"})  
with urllib.request.urlopen(req, timeout=120) as resp:  
    zip_bytes = resp.read()  
with zipfile.ZipFile(io.BytesIO(zip_bytes)) as zf:  
    with zf.open(target_csv) as src:  
        target.write_bytes(src.read())
```

4.4 Manejo de errores

La fase 0 maneja explícitamente tres tipos de fallas:

- **HTTPError**: el servidor responde con código de error (404, 500). Esto pasaría si el INEI cambia las URLs.
- **URLError**: no hay conexión a internet, el DNS falla, o el INEI está caído temporalmente.

- **BadZipFile:** el archivo descargado no es un ZIP válido (por ejemplo, si el INEI devuelve una página HTML de error).

Cada error muestra un mensaje claro al usuario y termina la ejecución. Nunca falla en silencio.

5. Fase 1 — Preprocesamiento de la ENAHO

5.1 El reto: seis archivos en distintos niveles

La ENAHO no es un solo archivo plano. Está dividida en módulos, y cada módulo tiene un **nivel de análisis** distinto:

- **Nivel persona:** una fila por persona individual (módulos 02 Miembros, 03 Educación, 05 Empleo).
- **Nivel hogar:** una fila por hogar (módulos 01 Vivienda, 18 Equipamiento, Sumaria).

Para hacer un análisis a nivel persona, debemos **'duplicar' la información del hogar** en cada miembro. Por ejemplo, si un hogar tiene internet, los cinco miembros del hogar reciben `INTERNET_HOGAR = 1`.

5.2 Las llaves del merge

Para combinar información de distintos módulos, necesitamos **llaves de identificación únicas**:

```
# Llave para identificar una persona
KEYS_PERSON = ["CONGLOME", "VIVIENDA", "HOGAR", "CODPERSONO"]

# Llave para identificar un hogar
KEYS_HOUSEHOLD = ["CONGLOME", "VIVIENDA", "HOGAR"]
```

■ Analogía

Piensa en estas llaves como un DNI compuesto. Para identificar a una persona única en el Perú no basta con su nombre: necesitas el DNI (ocho dígitos). Para identificar una persona en la ENAHO no basta con CODPERSONO (que solo dice 'persona 1, 2, 3 del hogar'): necesitas también el conglomerado, la vivienda y el hogar para que la combinación sea única en todo el país.

5.3 Construcción de variables clave

5.3.1 Tenencia de billetera digital

La pregunta P558E1 del módulo 5 es **multi-respuesta**: la persona puede marcar varias opciones (cuenta de ahorros, tarjeta de débito, billetera digital, etc.). Cada opción es una columna distinta. La opción 9 corresponde a billetera digital:

```
def detect_billetera_tenencia(m05):
    # P558E1_9 == "9" significa que la persona marcó la opción 9
    col = "P558E1_9"
    return (m05[col].astype(str).str.strip() == "9").astype(int)
```

5.3.2 Smartphone (formato long → wide)

El módulo 18 viene en formato **long**: una fila por equipo del hogar (cocina, refrigerador, smartphone, etc.). Para construir la variable SMARTPHONE necesitamos filtrar las filas correspondientes a smartphone (P612N == 10) y mirar si el hogar lo tiene (P612 == 1).

5.3.3 Ingreso per cápita

Construido a partir de la Sumaria como ingreso del hogar dividido entre los miembros:

```
sum_h["INGRESO_PC"] = (  
    pd.to_numeric(sum_h["INGHOG2D"], errors="coerce") /  
    pd.to_numeric(sum_h["MIEPERHO"], errors="coerce").replace(0, np.nan)  
)
```

5.4 Salida de la fase 1

El resultado es un único archivo: `data/clean/enaho_2024_clean.csv`, con 117,721 personas y 27 columnas. Listo para el análisis RDD.

6. Fase 2 — Limpieza para regresión discontinua

Antes de estimar el modelo, hay que preparar los datos siguiendo las reglas específicas del diseño RDD.

6.1 Coerción a numérico de la edad

Aquí encontramos un detalle sutil pero crítico. La columna EDAD en los CSV del INEI a veces viene con espacios en blanco para los valores faltantes. Si pandas la lee como texto, no podremos restar el umbral. Por eso la **forzamos a numérico**:

```
df[RUNNING_VAR_RAW] = pd.to_numeric(df[RUNNING_VAR_RAW], errors="coerce")
```

■■ Defensa contra datos sucios

Esta línea es un escudo defensivo. El parámetro `errors='coerce'` convierte a NaN cualquier valor que no se pueda interpretar como número. Sin esta línea, el script crashea con un error confuso varias líneas más abajo. **Anticipar el error en su origen** es buena práctica.

6.2 Centrar la running variable

En RDD trabajamos con la **running variable** centrada en el umbral. Si el umbral es 65 años, redefinimos:

```
df["running_centered"] = df["EDAD"] - 65
```

Ahora `running_centered = -3` significa tres años por debajo del umbral, y `running_centered = +5` significa cinco años por encima. El cero es justo el cutoff.

6.3 Construir el indicador de tratamiento

La regla del programa es: 65 años o más reciben el tratamiento.

```
df["treat"] = (df["running_centered"] >= 0).astype(float)
```

6.4 Validación: tabla de missingness

La fase 2 imprime una tabla con el porcentaje de valores faltantes para cada variable clave. Esto le permite al investigador detectar anomalías antes de estimar:

```
--- Missingness table (key variables) ---
EDAD                                :      0 missing (0.0%)
TIENE_BILLETERA                     :      0 missing (0.0%)
USA_BILLETERA                       :      0 missing (0.0%)
INTERNET_HOGAR                      :      0 missing (0.0%)
SMARTPHONE                          :      0 missing (0.0%)
POBREZA                             :      0 missing (0.0%)
NIVEL_EDUCATIVO                     : 11,102 missing (9.4%)
```

Si `NIVEL_EDUCATIVO` tuviera 50% de missing, sabríamos que algo está mal. Pero un 9.4% es esperable porque los menores de edad escolar todavía no tienen un nivel educativo registrado.

7. Fase 3 — Estimación principal del modelo

7.1 ¿Qué es regresión discontinua?

■ Concepto fundamental: RDD

Cuando una política asigna el tratamiento basándose en un **umbral arbitrario** de una variable continua (por ejemplo, 'eres elegible si tienes 65 años o más'), las personas que están **justo por debajo** del umbral son casi idénticas a las que están **justo por encima**, excepto por una sola cosa: una recibe el tratamiento y la otra no. Comparar sus resultados nos da el **efecto causal local** del tratamiento.

7.2 La función estrella: `run_rdd()`

Esta es la función más importante del script. Hace cuatro cosas, en orden:

- Filtra observaciones válidas (sin NaN en outcome y running variable).
- Intenta estimar usando `rdrobust`, la librería estándar desarrollada por Cattaneo, Calónico y Titiunik.
- Si `rdrobust` falla (por ejemplo, memoria insuficiente o muestra muy pequeña), cae en un **fallback** a `statsmodels.WLS` con kernel triangular.
- **Siempre** retorna un diccionario con todas las columnas pobladas, aunque algunas sean NaN. Esto evita que las tablas finales tengan celdas vacías.

7.3 Las tres especificaciones del paper

El script estima el modelo bajo tres especificaciones distintas:

Especificación	Covariables incluidas
Baseline	Ninguna — solo running variable
Con covariables básicas	INTERNET_HOGAR, SMARTPHONE
Covariables extendidas	+ POBREZA, INGRESO_PC, NIVEL_EDUCATIVO

Mostrar varias especificaciones es estándar en publicaciones académicas: si el efecto se mantiene en las tres, ganamos confianza en que no estamos forzando un resultado con una sola elección.

7.4 Análisis de heterogeneidad

Para cada variable moderadora (POBREZA, INTERNET_HOGAR, SMARTPHONE), la fase 3 divide la muestra en alta/baja por la mediana y corre RDD en cada grupo. Esto revela si el efecto del tratamiento es más fuerte en algún subgrupo específico.

8. Fase 4 — Pruebas de robustez

Aquí está la diferencia entre un buen paper y uno malo. Un buen paper RDD **nunca** presenta un solo resultado: presenta el resultado principal y muchas pruebas de que ese resultado no es una coincidencia ni un artefacto.

La fase 4 ejecuta nueve pruebas distintas:

#	Prueba	¿Qué verifica?
1	McCrary / rddensity	Que las personas no manipulen su edad para entrar al programa.
2	Outcomes placebo	Que el tratamiento no afecte variables que NO debería afectar.
3	Sensibilidad de bandwidth	Que el resultado sea estable a $h/2$, h óptimo y $2h$.
4	Donut-hole	Que el resultado se mantenga si excluimos observaciones muy cerca del cutoff.
4b	Heterogeneidad SISFOH	Si el efecto se concentra en pobres extremos (target de Pensión 65).
5	Balance de covariables	Que no haya saltos discontinuos en covariables al cutoff.
6	Polinomios	Comparar especificación lineal ($p=1$) vs cuadrática ($p=2$).
7	Cutoffs placebo	Que no haya 'efectos' significativos en cutoffs falsos (p.ej. 50 años).
8	Inferencia por permutación	500 permutaciones de cutoffs aleatorios para construir un p-valor empírico.
9	Corrección BH (FDR)	Ajuste del p-valor por testeo de hipótesis múltiples.

■ Lección pedagógica

Un solo resultado positivo no demuestra nada. Cualquier investigador honesto sabe que es relativamente fácil encontrar UN resultado significativo si torturas los datos lo suficiente. Lo difícil — y lo valioso — es que el resultado sobreviva intacto a una batería como esta. Por eso los *referees* de las revistas serias siempre preguntan: ¿qué pasa con el bandwidth $h/2$? ¿Qué pasa con donut-hole de 2 años? Si el efecto desaparece, no era un efecto real.

9. Fase 5 — Tablas y figuras del paper

Ahora viene la parte estética: convertir todos los CSVs de resultados en tablas LaTeX listas para insertar en el paper, y figuras en PNG/PDF de calidad de publicación.

9.1 Outputs generados

Output	Contenido	Ubicación
table_1_summary.tex	Estadísticas descriptivas (treated vs control)	paper/tables/
table_2_main_results.tex	Resultados RDD principales + heterogeneidad	paper/tables/
table_3_robustness.tex	Cuatro paneles de robustez	paper/tables/
table_4_covariate_balance.tex	Balance de covariables al cutoff	paper/tables/
figure_1_rdplot	Gráfico de regresión discontinua	paper/figures/
figure_2_bandwidth_sensitivity	Estimaciones a distintos bandwidths	paper/figures/
figure_mccrary_density	Histograma de la running variable	paper/figures/

9.2 Validación de tablas: prevenir el desastre

La función `_validate_table()` revisa cada tabla generada antes de guardarla. Busca específicamente:

- La palabra 'nan' (resultado de un cálculo con NaN no manejado).
- Celdas vacías (patrones como `& &`).
- Última columna vacía (patrones como `& \`).

Si encuentra problemas, los reemplaza automáticamente con `- - -`. Una sola celda vacía en una tabla LaTeX puede romper toda la compilación del paper, así que esta validación es crítica.

10. Cómo correr el pipeline

10.1 Requisitos

- **Git** (para clonar el repositorio).
- **Python 3.9 o superior** (con pip incluido, que es lo normal).
- **Conexión a internet** (para descargar de INEI y auto-instalar dependencias la primera vez).

10.2 El comando único

```
$ git clone <url-del-repositorio>
$ cd pension-65-digital-wallet
$ python scripts/script_completo/script.py
```

Eso es todo. El script se encarga del resto:

- Detecta dependencias faltantes y las instala con pip.
- Descarga 511 MB de microdatos desde el INEI.
- Ejecuta las fases 1 a 5 en orden.
- Genera 13 outputs en sus ubicaciones correctas.

10.3 Tiempo estimado

En una computadora razonable (8 GB de RAM, conexión doméstica), el pipeline completo demora aproximadamente:

Fase	Tiempo aproximado
0 — Descarga INEI	1–3 minutos (depende del internet)
1 — Preprocess	2–4 minutos (carga 511 MB en pandas)
2 — Cleaning	10–30 segundos
3 — Estimación	30–60 segundos
4 — Robustez	1–2 minutos (incluye 500 permutaciones)
5 — Tablas y figuras	10–20 segundos
Total	Aproximadamente 5–10 minutos

11. Glosario de conceptos clave

RDD (Regression Discontinuity Design)

Diseño de regresión discontinua. Método cuasi-experimental que explota umbrales arbitrarios de elegibilidad para identificar efectos causales locales.

Running variable

La variable continua que define el umbral del tratamiento. En este estudio: la edad del individuo. Centrada en el cutoff (65 años) para que el umbral quede en cero.

Cutoff o umbral

El valor de la running variable que separa tratados de controles. Aquí: 65 años.

Bandwidth (h)

El ancho de la ventana alrededor del cutoff que se usa en la estimación. Bandwidth chico produce más identificación pero menos potencia estadística; bandwidth grande, lo opuesto.

MSE-optimal bandwidth

El bandwidth que minimiza el error cuadrático medio. La librería `rdrobust` lo calcula automáticamente con la opción `bwselect='mserd'`.

Kernel triangular

Función de pesos que da más importancia a observaciones cercanas al cutoff y menos a las lejanas. Es el kernel estándar en RDD.

Placebo

Variable o cutoff falso que NO debería ser afectado por el tratamiento. Si encontramos un 'efecto' en un placebo, hay algo mal con nuestro diseño.

Donut-hole

Especificación que excluye observaciones muy cercanas al cutoff. Útil cuando hay sospecha de manipulación o errores de medición justo en el umbral.

McCrary test

Prueba estadística que detecta saltos discontinuos en la densidad de la running variable al cutoff. Un salto sugiere manipulación (personas alterando su edad para entrar o evitar el programa).

Sumaria (ENAH)

Módulo de la ENAH que contiene variables ya calculadas a nivel hogar: ingreso total, gasto total, miembros, clasificación de pobreza (POBREZA), entre otras.

SISFOH

Sistema de Focalización de Hogares. Clasifica a los hogares en extrema pobreza, pobreza no extrema y no pobreza. La variable POBREZA en la Sumaria refleja esta clasificación.

12. Conclusiones

Después de recorrer las cinco fases, podemos resumir lo que hace este script y por qué su diseño es importante:

12.1 Lo que logramos

- **Un solo punto de entrada:** `python script.py`.
- **Cero dependencias manuales:** el script descarga datos, instala paquetes, genera todos los outputs.
- **Robusto frente a fallas:** maneja errores de red, datos sucios y fallas de estimadores con fallbacks.
- **Completo:** del dato crudo del INEI hasta las tablas LaTeX listas para el paper.
- **Determinístico:** la misma semilla aleatoria siempre da los mismos resultados.

12.2 La filosofía detrás

■ Reflexión final

Un paper de investigación cuyo código no se puede reproducir no es un aporte científico: es un acto de fe. Este script convierte el aporte científico en algo verificable por cualquiera con Python instalado y conexión a internet.

12.3 Para seguir aprendiendo

Si este documento te interesó y deseas profundizar, te recomiendo explorar:

- La documentación de `rdrobust` (Cattaneo, Calónico, Titiunik) — el estándar de oro en RDD.
- El libro '*A Practical Introduction to Regression Discontinuity Designs*' (Cattaneo, Idrobo, Titiunik), disponible gratuitamente en SSRN.
- La ENAHO en general: el INEI publica un manual del encuestador muy útil para entender cómo se construye cada variable.
- La filosofía de *literate programming* de Donald Knuth: el código debe ser legible para humanos antes que ejecutable por máquinas.

Espero que esta guía te haya servido para entender no solo lo que el código hace, sino también por qué fue diseñado de esta manera. La buena investigación no es solo encontrar un resultado: es construir un proceso que cualquiera pueda verificar.